

fun.lol Product Design Document

Generated: 2026-05-24

Source inspection scope: 'index.html', 'script.js', 'styles.css', 'server.js', 'package.json', and 'supabase-schema.sql'.

1. Executive Summary

fun.lol is a customizable public profile platform where users can create a public bio page, add visual and audio media, connect with friends, join tribes, chat with tribe members, and play mini-games from a private dashboard. The current product blends a personal identity page with lightweight social and entertainment features.

2. Product Vision

The long-term vision is for fun.lol to become a highly personalized social identity hub. Users should be able to express themselves through media-rich profiles, share one public link, keep up with friends, create small communities, and enjoy interactive experiences without leaving the platform.

3. Target Users

- Creators who want a personalized profile link.
- Gamers who enjoy expressive profiles and mini-games.
- Social users who want a lightweight friend graph.
- Friend groups who want private-ish spaces around shared identity.
- Community or tribe members who want a shared group and chat.
- Public visitors who open a user's profile link.

4. Problem Statement

Many profile-link tools are static, generic, and disconnected from the user's actual social experience. fun.lol solves this by combining profile customization, media, friend requests, tribe communities, tribe chats, and mini-games in one lightweight web platform.

5. Product Goals

- Enable personal profile customization.
- Support public profile sharing by handle.
- Make social discovery and friend connection simple.
- Provide tribe/community engagement.
- Add mini-game entertainment inside the dashboard.
- Preserve a strong dark, animated, profile-first brand feel.

6. Non-Goals

The current version does not include:

- A full social media feed.
- Payment or subscription systems.
- Advanced moderation dashboard.
- Native mobile app.
- Enterprise admin portal.
- End-to-end encrypted messaging.

- Realtime chat infrastructure.
- Password reset or email verification.

7. Key Features

Account & Authentication

- Email and password sign up.
- Login with saved browser session.
- Auto-login for returning users.
- Server-side password hashing.
- Settings page with account and public profile details.

Dashboard

- Private logged-in home screen.
- Sidebar navigation for Bio, Games, Tribes, and Settings.
- Collapsible sidebar.
- Friends and notifications widgets.
- Dashboard theme selector.
- Pointer style and color controls.
- Mute Outside Bio control.

Bio/Profile Builder

- Editable display name, handle, bio, and location.
- Public profile publishing.
- Visitor preview.
- Public link generation.
- Profile view counter.
- Profile card with avatar, UID, featured section, location, views, and social buttons.

Profile Customization

- Avatar upload.
- Background image or video upload.
- Background music upload.
- Remove background and music files.
- Theme selection.
- Animated background, compact links, and dark video overlay toggles.
- Dot cursor, cursor color, and sparkle effects.

Music & Entry Experience

- Public profile entry screen before the profile opens.
- Music starts after user interaction.
- Music controls on the owner's bio editor.
- Public visitors do not see the full music information box.
- Public video loop transition with audio fade behavior.

Social Links

- Discord, Instagram, TikTok, YouTube, X, and GitHub social fields.
- Profile social output uses icons rather than plain text labels.

Friends & Notifications

- Add friends by name, handle, or profile link.
- Send and accept friend requests.
- Remove friends after confirmation.
- View sent friend requests.

- Notifications widget and friend request tab.
- Periodic refresh of friend and notification state.

Tribes

- Add Friends, Your Friends, Your Tribes, Join a Tribe, Your Tribe Chats, and Friend Requests tabs.
- Create tribes with a name and color.
- Invite friends to tribes.
- Accept or decline tribe invites.
- Search for tribes.
- Request to join tribes.
- Owner approval for join requests.
- Owner-only rename, color change, delete, and member removal.

Tribe Chats

- Grid of chats for tribes the user can access.
- Open a selected tribe chat.
- Send messages scoped to that tribe.
- Exit chat back to the grid.
- Non-members are blocked by the backend.

Games

- Games dashboard with card grid and expanded game views.
- Snake with server-backed high score.
- Click Rush with local best score.
- Wordle with English dictionary validation from 'words-5.txt'.
- Crossy Road-style game with increasing difficulty and obstacles.
- Memory Match and Orbit Dodge placeholders.

Backend/Data

- Node.js backend using built-in 'http'.
- Local JSON fallback storage.
- Optional Supabase database support.
- Optional Supabase Storage media bucket.
- API routes for auth, profiles, friends, tribes, tribe chats, media, games, sitemap, and robots.

Responsive UX

- Mobile layout adjustments.
- Custom cursor disabled on coarse pointer/mobile devices.
- Responsive dashboard/sidebar behavior.
- Glassmorphism dark UI.
- Animated star/void-style background.
- Hover movement effects.
- Loading, welcome, and entry transitions.

8. Functional Requirements

Authentication

1. The system shall allow a user to sign up with email and password.
1. The system shall reject invalid emails and passwords shorter than the configured minimum.
1. The system shall hash passwords before storing them.
1. The system shall allow a user to log in with a valid email and password.
1. The system shall return a session token after successful signup or login.
1. The system shall load account metadata through 'GET /api/me'.

Dashboard

- 1. The system shall show the authenticated dashboard only after login.
- 1. The system shall provide Bio, Games, Tribes, and Settings navigation.
- 1. The system shall allow the sidebar to collapse and expand.
- 1. The system shall persist dashboard theme, cursor mode, cursor color, mute setting, and sidebar state in localStorage.
- 1. The system shall show friends and notifications widgets on the home screen.

Profile Builder

- 1. The system shall allow users to edit name, handle, bio, and location.
- 1. The system shall normalize handles before publishing.
- 1. The system shall prevent authenticated users from editing profiles owned by another user.
- 1. The system shall generate '/u/:handle' public profile paths.
- 1. The system shall increment public views when a profile is opened with '?view=1'.

Media

- 1. The system shall allow avatar, background, and music upload from the browser.
- 1. The system shall preview selected media before publish.
- 1. The system shall store media in Supabase Storage when Supabase is configured.
- 1. The system shall serve media through profile media API endpoints.
- 1. The system shall allow background and music files to be removed.

Friends and Notifications

- 1. The system shall send friend requests to valid published profile handles.
- 1. The system shall prevent self friend requests.
- 1. The system shall prevent duplicate friendships from being added.
- 1. The system shall allow recipients to accept friend requests.
- 1. The system shall remove accepted requests from pending lists.
- 1. The system shall allow friends to be removed after confirmation.
- 1. The system shall refresh friend and notification state periodically.

Tribes

- 1. The system shall allow authenticated profile owners to create tribes.
- 1. The system shall allow tribe owners to invite existing friends.
- 1. The system shall show tribe invites as notifications.
- 1. The system shall allow invited users to accept or decline.
- 1. The system shall allow users to search tribes by name.
- 1. The system shall allow users to request to join a tribe.
- 1. The system shall allow tribe owners to accept or decline join requests.
- 1. The system shall allow owners to rename tribes and change tribe color.
- 1. The system shall allow owners to delete tribes.
- 1. The system shall allow owners to remove members except themselves.

Tribe Chats

- 1. The system shall show chat cards only for tribes the user owns or belongs to.
- 1. The system shall load messages for the selected tribe only.
- 1. The system shall reject chat access for non-members.
- 1. The system shall reject empty chat messages.
- 1. The system shall append sent messages to the selected tribe chat.
- 1. The system shall keep chat state when exiting back to the chat grid.

Games

- 1. The system shall allow users to open games from a card grid.

1. The system shall expand the selected game and provide a return action.
1. The system shall show how-to-play content for each playable game.
1. The system shall save Snake high score through the backend.
1. The system shall save Click Rush and Crossy Road best scores locally.
1. The system shall validate Wordle guesses against a dictionary.

9. Non-Functional Requirements

Security:

- Passwords must be hashed before storage.
- Private APIs must require valid sessions.
- Owner-only actions must be checked on the server.

Performance:

- Public profiles should minimize payload size by avoiding inline media in public view mode when storage paths exist.
- Background videos should load without blocking core profile text.

Scalability:

- Production should use Supabase or another managed database instead of local JSON.
- High-volume data such as chat messages should move to dedicated tables.

Availability:

- The app can run locally with JSON storage and in hosted mode with Supabase.
- Recommendation: add backups and monitoring for production.

Responsiveness:

- Dashboard and profile layouts should adapt to mobile and desktop.
- Custom cursor effects should be disabled on coarse pointer devices.

Accessibility:

- Interactive controls should preserve labels or accessible names.
- Recommendation: add keyboard and screen reader audits for games and dialogs.

Maintainability:

- Recommendation: split the growing 'server.js' and 'script.js' into modules as the product grows.

Browser compatibility:

- The app should work in modern Chromium, Edge, Safari, and Firefox browsers.
- Recommendation: test media autoplay behavior across browsers because music requires user interaction.

Data privacy:

- Public profile payloads should not include password hashes, session tokens, or private owner fields.
- Recommendation: avoid exposing Supabase service keys to the client.

Error handling:

- API errors should return clear messages.
- UI should show toast or status messages for failed actions.

10. User Roles & Permissions

Role: Public visitor; Description: Not logged in, opens a public profile; Key permissions: View public profile and media entry experience.

Role: Logged-in user; Description: Authenticated account holder; Key permissions: Access dashboard, publish own profile, use games, manage friends and tribes.

Role: Friend; Description: User connected through accepted friend request; Key permissions: Appears in friends list and can be invited to tribes.

Role: Tribe member; Description: User included in a tribe member list; Key permissions: View tribe details and access tribe

chat.

Role: Tribe owner; Description: Creator/owner of a tribe; Key permissions: Rename, recolor, delete tribe, approve joins, remove members, access chat.

Role: Invited user; Description: User who received a tribe invite; Key permissions: Accept or decline the invite.

Role: Join requester; Description: User who requested tribe membership; Key permissions: Wait for owner approval; cannot access chat until accepted.

11. UX/UI Principles

- Dark glassmorphism visual language.
- Animated star/void background as a core brand element.
- Compact dashboard layout with clear navigation.
- Profile-first personalization with live preview behavior.
- Smooth loading, welcome, and entry transitions.
- Game-like interaction patterns for cards, hover motion, and cursor effects.
- Mobile-friendly behavior with custom cursor effects disabled where inappropriate.

12. Data Model

Object: User; Key fields: id, email, passwordHash/password_hash, profileHandle, profilePath, profileUrl, snakeHighScore; Current persistence: 'users.json' or 'app_users'.

Object: Profile; Key fields: handle, ownerUserId, name, bio, location, theme, views, updatedAt; Current persistence: 'profiles.json' or 'app_profiles.data'.

Object: AccountSettings; Key fields: dashboardTheme, cursorMode, cursorColor, muteOutsideBio, sidebarCollapsed; Current persistence: Browser localStorage.

Object: SocialLink; Key fields: platform, value, normalizedUrl; Current persistence: Profile JSON.

Object: FriendRequest; Key fields: id, fromName, fromHandle, fromLink, createdAt; Current persistence: Target profile JSON.

Object: Friendship; Key fields: id, name, handle, link; Current persistence: Profile JSON.

Object: Notification; Key fields: derived type, actor, target, createdAt; Current persistence: Derived from friend requests, tribe invites, and join requests.

Object: Tribe; Key fields: tribelId, name, ownerId, ownerDisplayName, ownerHandle, memberIds, pendingInviteIds, pendingJoinIds, themeColor, messages, createdAt, updatedAt; Current persistence: Owner profile JSON.

Object: TribeMember; Key fields: userId, displayName, handle, link; Current persistence: Derived from memberIds and profile data.

Object: TribeInvite; Key fields: id, tribelId, tribeName, ownerId, ownerDisplayName, ownerHandle, createdAt; Current persistence: Invited user's profile JSON.

Object: TribeJoinRequest; Key fields: id, tribelId, tribeName, requesterId, requesterDisplayName, requesterHandle, createdAt; Current persistence: Owner profile JSON.

Object: TribeMessage; Key fields: id, senderId, senderDisplayName, senderHandle, text, createdAt; Current persistence: Tribe object in owner profile JSON.

Object: GameScore; Key fields: userId, game, highScore; Current persistence: Snake in user storage; other best scores in localStorage.

Object: MediaAsset; Key fields: avatar/background/music data or path, name, type; Current persistence: Profile JSON plus optional Supabase Storage object.

13. Key User Flows

- Sign up: User enters email/password, server creates account and session, dashboard loads.
- Login: User enters credentials, server verifies password, session token is stored.
- Edit bio profile: User changes profile fields, preview updates, user publishes.
- Upload media: User selects avatar/background/music, preview updates, profile publish stores media.
- Publish public profile: Server saves profile and returns '/u/:handle'.
- Visitor opens profile: Visitor enters profile page, view count increments, public profile loads.
- Add friend: User enters a handle/link, server sends request to target profile.
- Accept friend request: Recipient accepts, both friend lists update.

- Create tribe: Owner creates tribe, optionally invites friends.
- Invite friend to tribe: Friend receives tribe invite notification and accepts/declines.
- Join tribe: User searches tribe and sends join request.
- Approve join request: Owner accepts requester into member list.
- Open tribe chat: Member selects chat card and loads scoped messages.
- Send tribe message: Member sends non-empty text, message is appended to tribe.
- Play game: User opens game, plays, game-over state shows score.
- Save score: Snake score saves through backend; selected other games use localStorage.
- Change dashboard/theme/cursor settings: User changes controls and settings persist locally.

14. Edge Cases

- Duplicate handle: Server rejects editing a profile owned by another account.
- Invalid profile link: Friend request route returns an error for invalid target handles.
- Failed media upload: Server returns an error and profile publish should show a failure message.
- Removed background/music file: UI clears local state and publish persists removal.
- Empty bio fields: Defaults are applied for name, handle, bio, and location.
- Friend already added: Existing friendship prevents duplicate request behavior.
- Duplicate friend request: Existing pending request is reused.
- Tribe already exists: Current implementation does not enforce unique tribe names. Recommendation: add duplicate-name handling per owner.
- User already in tribe: Join route returns joined status.
- User already invited: Duplicate invite records are avoided.
- Non-member accessing tribe chat: Server returns a forbidden error.
- Owner removing themselves: Server rejects removing the owner.
- Tribe deleted while chat is open: Active chat is cleared when tribe state refresh removes access.
- Empty chat message: Server rejects empty sanitized text.
- Game score save failure: Snake falls back to local best-score behavior in the frontend.
- Local JSON storage unavailable: Server file read/write would fail. Recommendation: add operational monitoring and backups.
- Supabase unavailable: API requests will fail in Supabase mode. Recommendation: add retry and user-friendly status handling.

15. Acceptance Criteria

Authentication:

- A new user can sign up and reaches the dashboard.
- Existing users can log in without recreating accounts.
- Invalid credentials produce a clear error.

Dashboard:

- Sidebar navigation changes active sections.
- Sidebar collapse state persists.
- Cursor, theme, and mute controls do not overlap and remain clickable.

Profile:

- A user can publish a profile and receive a public link.
- Visitors can open '/u/:handle' without edit controls.
- View count increases when a public profile is viewed.

Media:

- Avatar, background, and music can be selected, previewed, saved, and loaded again.
- Remove buttons clear background and music.

Friends:

- A user can send a friend request by handle or profile link.
- Recipient can accept the request.
- Friends appear without requiring a full page refresh after periodic refresh.
- Removing a friend asks for confirmation.

Tribes:

- A user can create a tribe and invite friends.
- Invited users receive Yes/No notifications.
- Users can search and request to join tribes.
- Owners can approve join requests.
- Owners can rename, recolor, delete, and remove members.
- Non-owners do not see or use owner controls.

Tribe Chats:

- Members see chat cards for their tribes.
- Non-members cannot open a tribe chat.
- Messages are scoped to the active tribe.
- Exit Chat returns to the chat grid.

Games:

- Playable games open from the grid.
- Snake, Click Rush, Wordle, and Crossy Road show game-over states.
- Wordle rejects guesses not found in the dictionary.
- Snake high score is saved for the authenticated user.

16. Analytics & Success Metrics

- Signups.
- Logins and returning users.
- Published profiles.
- Public profile views.
- Media uploads.
- Profile publish success rate.
- Friend requests sent and accepted.
- Active friendships.
- Tribes created.
- Tribe invites accepted/declined.
- Tribe join requests accepted/declined.
- Tribe chat messages sent.
- Games opened.
- Games completed.
- Snake high score saves.
- Theme/cursor customization usage.

17. Risks & Mitigations

Risk: Local JSON storage may not scale; Type: Technical; Mitigation: Move production data to Supabase tables and add backups.

Risk: Chat stored inside owner profile JSON can become large; Type: Technical; Mitigation: Move messages to a dedicated messages table or realtime service.

Risk: No session expiration visible; Type: Security; Mitigation: Add token expiry, refresh, and logout invalidation.

Risk: Public media may be large; Type: Performance; Mitigation: Add upload limits, compression, and CDN-backed storage.

Risk: User-generated content can be abusive; Type: Product/Safety; Mitigation: Add reporting, moderation, block lists, and content rules.

Risk: Music/video autoplay varies by browser; Type: UX; Mitigation: Keep entry gate and test browser-specific media behavior.

Risk: Large 'script.js' and 'server.js' reduce maintainability; Type: Engineering; Mitigation: Split by feature modules and add automated tests.

18. Future Enhancements

- Realtime tribe chat.
- Push notifications.
- Image/audio attachments in chat.
- Tribe privacy controls.
- Moderation tools and report flows.
- Profile templates.
- More games.
- Public tribe discovery.
- Supabase-first production database with normalized social tables.
- Mobile app.
- Premium customization features.
- Password reset and email verification.
- Admin analytics dashboard.